

Cost of Hybrid Quantum PINNs: Parameter Count vs. Wall-Clock Time

Author(s) to be added
Affiliation to be added

Abstract

Hybrid quantum–classical physics-informed neural networks (QCPINNs) are usually reported to be efficient since they have fewer trainable parameters than a classical PINN. We ask how much wall-clock time this saving costs. We evaluate a classical PINN together with four hybrid configurations that vary in qubit count and circuit depth on a one-dimensional seawater temperature diffusion benchmark. *Parameter count:* the quantum branch adds only 16–64 trainable parameters, and the hybrids have 2421–2717 parameters in total against 4353 for the classical baseline. This amounts to a reduction of 38–44%, almost all of it due to the smaller classical portion. *Timing:* relative to the classical baseline, the four hybrid configurations are 14–59× slower on the forward pass, 14–102× slower for the first-order gradient, and 13–94× slower for the second-order gradient, growing with qubit count, circuit depth and batch size. *Takeaway:* the number of parameters alone tends to overstate the efficiency of hybrid models. This paper is not about accuracy.

1 Introduction

Physics-informed neural networks (PINNs) [18] solve forward and inverse partial differential equation (PDE) problems by embedding the governing equation, boundary conditions and, for inverse problems, observations into the neural network’s training loss. We study a one-dimensional temperature diffusion problem, the seawater temperature equation of Han et al. [8], under Dirichlet, Neumann and Robin boundary conditions. Fast, differentiable surrogates for this equation are attractive for energy-aware control of engineered water bodies, such as swimming pools or other managed basins, where a model that returns temperature and its gradients cheaply can drive heating, circulation or shading decisions in an optimal-control or model-predictive-control loop.

Hybrid quantum–classical PINNs (QCPINNs), in which part of the network is replaced by a variational quantum circuit (VQC), have been reported to match or exceed classical PINNs while using far fewer trainable parameters [7]. Parameter count has now become the standard efficiency metric in this literature.

Gap. Wall-clock cost is rarely the headline metric and, where it is reported, it is a single aggregate figure (see Related work below). Yet near-term VQCs are carried out using classical simulation, since its cost increases with circuit width and depth (Section 2). Training a PINN also requires first- and second-order derivatives of the network with respect to its *inputs*, which are more expensive than a plain forward pass and interact with the simulated circuit.

Research questions.

RQ1 How large is the parameter-count advantage of hybrid models, and where does it come from?

RQ2 What timing cost accompanies it, and how does that cost scale with qubit count, circuit depth, batch size and simulator backend?

Contributions.

1. A parameter-count breakdown of a classical PINN and four hybrid configurations under a common accounting.
2. Stage-wise timing of the forward pass, the first-order input gradient and the second-order input gradient.
3. A sweep of qubit count, circuit depth, batch size and simulator backend.

Accuracy comparisons, gradient-norm ablations and barren-plateau discussion are not the subject of this paper.

Outline. Section 2 defines the PINN loss, the timed stages and the VQC basics used throughout; Section 3 describes the classical baseline and the four hybrid configurations; Section 4 details the timing protocol and the size-matched classical controls. This draft covers the setup only – the parameter-count and timing results, discussion and conclusion follow in later sections of the full paper.

Related work. Hybrid quantum PINNs have been proposed for fluid flow, wave, Helmholtz and Klein–Gordon problems [7, 12], reservoir seepage [19], hydrological forecasting [9] and Maxwell’s equations [5], with theoretical work on universal approximation and barren-plateau mitigation [11]. The headline efficiency claim in this literature is parameter count: QCPINNs are reported to reach comparable accuracy with 10–30% of the parameters of a classical PINN [7], hybrid models with fewer parameters than the best classical model [12, 22], and about 44% fewer parameters together with roughly three times fewer training epochs [9].

Cost is acknowledged but usually reported as one aggregate figure. Farea et al. [7] report an increase of 15- to 40-fold in time per training iteration relative to classical PINNs, at similar memory. Leong et al. [12] state that simulating the quantum layers imposes an overhead of at least two orders of magnitude for the same number of trainable parameters, but compare training efficiency through epochs rather than wall-clock time. Trahan et al. [22] find a quantum PINN to take nearly four times as long as a classical PINN with a similar number of parameters on a 2D Poisson problem. These figures differ widely and depend on hardware, simulator, circuit and batch size; none of them, to our knowledge, breaks the cost down into forward, first-order and second-order input-gradient stages, or varies qubit count, depth, batch size and backend in a controlled way. On the simulation side, Chen et al. [5] develop a GPU-accelerated simulation library for computing circuit outputs and their derivatives, to make QPINN training practical. This paper fills the gap with a controlled timing study alongside the parameter count.

2 Background and Key Concepts

2.1 PINN loss structure

We consider the diffusion equation

$$\frac{\partial T}{\partial t} = k_z \frac{\partial^2 T}{\partial z^2}, \quad z \in [0, 2], \quad t \in [0, 5], \quad (1)$$

with diffusivity $k_z = 0.05$ in forward problems and k_z estimated from data in inverse problems. Equation (1) says how fast the temperature changes in *time* (left) equals the diffusivity times its curvature in *depth* (right); checking whether a network’s prediction obeys it means computing the derivatives each side needs, which is what Section 2.2 times. Let $\hat{T}_\theta(z, t)$ be the network output with trainable parameters θ and input $x = (z, t)$. The PINN loss combines four mean-squared terms,

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PDE}} + \mathcal{L}_{\text{BC}} + \mathcal{L}_{\text{IC}} + \mathcal{L}_{\text{data}}, \quad (2)$$

namely the PDE residual at interior collocation points, the boundary-condition residual, the misfit to the initial condition, and, for inverse problems only, the misfit to observations; all terms have unit weight.

Initial and boundary conditions. Closed-form solutions exist for all three boundary-condition types. Table 1 lists the conditions imposed. The initial condition is the exact solution at $t = 0$; for the Dirichlet problem the exact solution vanishes at both boundaries, so $\mathcal{L}_{\text{BC}}$ penalises $\hat{T}_\theta(0, t)$ and $\hat{T}_\theta(2, t)$ directly. The Neumann and Robin conditions involve $\partial\hat{T}_\theta/\partial z$ at the boundaries and therefore already require a first-order input gradient (Section 2.3).

Table 1: Initial and boundary conditions of the three benchmark problems. Each boundary condition holds at $z = 0$ and $z = 2$ for all $t \in [0, 5]$.

Type	Boundary condition	Initial condition $T(z, 0)$
Dirichlet	$T = 0$	$\sin \frac{\pi z}{2} + \sin \pi z$
Neumann	$\partial T/\partial z = 0$	$\cos \frac{\pi z}{2} + \cos \pi z$
Robin	$\partial T/\partial z + T = 0$	$e^{-z} + \frac{\pi}{2} \cos \frac{\pi z}{2} - \sin \frac{\pi z}{2} + \pi \cos \pi z - \sin \pi z$

Training points. We sample 2540 interior collocation points uniformly in $[0, 2] \times [0, 5]$, 40 points on each of the two boundaries, and 160 points at $t = 0$. Inverse problems additionally use 12 observations on a uniform 4×3 grid in (z, t) .

2.2 Four cumulative stages

Training and evaluating a PINN involves four computations of increasing cost, each reusing the computation graph of the one before it. We name them here as they appear in the timing results (Section 4) and summarise their role before deriving each in detail below.

forward evaluates the network once, $\hat{T}_\theta(z, t)$, with no derivatives at all – the cost of using the trained network to make a single prediction.

first_grad additionally computes the *first-order input gradient* $\nabla_x \hat{T}_\theta = (\partial\hat{T}_\theta/\partial z, \partial\hat{T}_\theta/\partial t)$ (Section 2.3), needed for the time-derivative term on the left of Eq. (1) and for the Neumann and Robin boundary residuals.

second_grad additionally computes the *second-order input gradient* $\partial^2\hat{T}_\theta/\partial z^2$ (Section 2.4), the diffusion term on the right of Eq. (1). Only once this is available can the PDE residual r_θ (below) be evaluated at all, so **second_grad** is the cost of *checking* whether a prediction satisfies the governing equation.

param_grad additionally computes the *parameter gradient* $\partial\mathcal{L}_{\text{PDE}}/\partial\theta$ (Section 2.5), obtained by treating r_θ as part of the loss and backpropagating once more, to the network’s weights rather than to its input. Where the first three stages only check the network, **param_grad** is what the optimiser actually runs at every training step to *correct* it.

Because the stages are cumulative, **second_grad** already includes the work of forward and **first_grad**; a ratio such as the **second_grad-to-forward** ratio is therefore the cost of *everything* the PDE residual needs, relative to a single plain prediction, not the cost of the second derivative in isolation.

2.3 First-order input gradient

The PDE residual is

$$r_\theta(z, t) = \frac{\partial\hat{T}_\theta}{\partial t} - k_z \frac{\partial^2\hat{T}_\theta}{\partial z^2}. \quad (3)$$

Evaluating it requires the *first-order gradient* of the output with respect to its input, $\nabla_x\hat{T}_\theta = (\partial\hat{T}_\theta/\partial z, \partial\hat{T}_\theta/\partial t)$, which PINNs obtain by automatic differentiation of the network [1, 13, 18]. It supplies the time derivative in r_θ and the boundary residuals of the Neumann ($\partial\hat{T}_\theta/\partial z = 0$) and Robin ($\partial\hat{T}_\theta/\partial z + \hat{T}_\theta = 0$) problems. This is the **first_grad** stage of Section 2.2. It must be distinguished from the *parameter gradient* $\partial\mathcal{L}/\partial\theta$ used by the optimiser (Section 2.5), which is a further backward pass through the graph of the input gradients.

2.4 Second-order input gradient

The diffusion term requires $\partial^2\hat{T}_\theta/\partial z^2$, obtained by differentiating $\partial\hat{T}_\theta/\partial z$ once more with respect to z . We compute both derivatives by reverse-mode automatic differentiation [1] in PyTorch [16], retaining the computation graph after the first pass so that it can be differentiated again; differentiating through a gradient with respect to the input in this way is known as double backpropagation [6]. The second-order gradient therefore costs a second backward pass through the graph created by the first, on top of the forward pass and the first-order pass. This defines the **second_grad** stage of Section 2.2: once it is available, r_θ can be formed and squared into $\mathcal{L}_{\text{PDE}}$.

2.5 Parameter gradient

Neither $\nabla_x\hat{T}_\theta$ nor $\partial^2\hat{T}_\theta/\partial z^2$ is what the optimiser uses to update the network: it needs the gradient of the loss with respect to the network’s own trainable parameters, $\partial\mathcal{L}/\partial\theta$. Restricting to the PDE term, $\mathcal{L}_{\text{PDE}} = \frac{1}{N} \sum_i r_\theta(z_i, t_i)^2$, computing $\partial\mathcal{L}_{\text{PDE}}/\partial\theta$ means backpropagating through r_θ itself – through the graph already built for the first- and second-order input gradients – one more time, to θ rather than to x . This is the **param_grad** stage: approximately the cost of the PDE term in one optimiser step, and not to be confused with the *input* gradients of Sections 2.3–2.4, which check the network rather than update it. The boundary, initial-condition and (for inverse problems) data terms of $\mathcal{L}$ in Eq. (2) contribute further, cheaper backward passes of their own that we do not separately time, so a full training step costs somewhat more than **param_grad**.

2.6 Variational quantum circuits

A VQC acts on n qubits and consists of an encoding of classical features into gate angles followed by L layers of parametrised single-qubit rotations and entangling gates; its outputs are expectation values of observables [2, 4]. The trainable quantum parameters are the rotation angles; n is the *width* and L the *depth* of the circuit. The encoding gates determine which functions of the input

the circuit can represent [21]; we use angle embedding (Section 3). VQCs are the model class of the noisy intermediate-scale quantum (NISQ) era [17], but in practice they are mostly run on classical simulators, and so are ours; we do not use quantum hardware. A statevector simulator stores 2^n complex amplitudes and, in the worst case, applies each gate in $O(2^n)$ operations, so memory and time grow exponentially with n and linearly with the number of gates, hence with the depth L [23].

2.7 How circuit gradients are obtained

On a classical simulator, as in our implementation, gradients are obtained by backpropagation: each backward pass traverses the operations that simulate the circuit, so the cost of every stage grows with the simulated circuit. On quantum hardware this is not possible, because intermediate quantum states cannot be measured and stored for reuse without destroying them [20]. Gradients of circuit outputs are instead obtained with the parameter-shift rule [15, 20]: for a gate generated by a Pauli operator, the derivative with respect to its angle is computed from two evaluations of the circuit with that angle shifted. Higher-order derivatives follow by iterating the rule, with up to 2^d shifted evaluations per derivative entry of order d (four for the Hessian) [14]. The same rule applies to the encoding gates, which gives derivatives of a quantum model with respect to its input [10]. In our hybrid network the circuit input is produced by classical layers, so input derivatives of the full network combine the chain rule through those layers with derivatives of the circuit. We measure the simulator case only and make no claim that simulator ratios carry over to hardware.

2.8 Notation

Table 2: Notation and terminology.

Symbol / term	Meaning
$x = (z, t)$	network input: depth and time
$\hat{T}_\theta$	network output, temperature, with parameters θ
k_z	thermal diffusivity
$\nabla_x \hat{T}_\theta$	first-order input gradient (first_grad stage)
$\partial^2 \hat{T}_\theta / \partial z^2$	second-order input gradient (second_grad stage)
$\partial \mathcal{L}_{\text{PDE}} / \partial \theta$	parameter gradient (param_grad stage; the optimiser’s own backward pass)
n, L	number of qubits (width), number of circuit layers (depth)
forward, first_grad, second_grad, param_grad	the four cumulative timed stages (Sections 2.2, 4)

3 Model Architectures

3.1 Classical baseline

The classical PINN is a fully connected network with five hidden layers of width 32 and tanh activations, mapping $(z, t) \mapsto \hat{T}_\theta$; it has 4353 trainable parameters.

3.2 Hybrid architecture

The hybrid network (Figure 1) replaces the middle of the network with a VQC: a classical pre-processor $2 \rightarrow 32 \rightarrow 32 \rightarrow n$ (tanh activations) maps the normalised input to n features, the

n = number of qubits, L = number of variational layers; tensors between blocks have shape $[batch, n]$

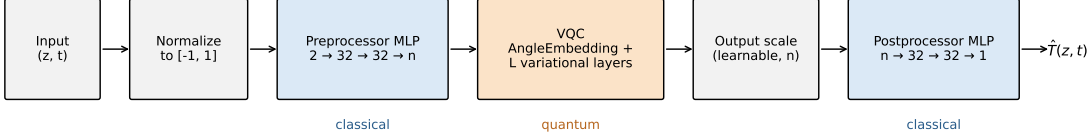


Figure 1: Hybrid QCPINN: classical layers, quantum layer, classical layers.

Table 3: Configurations. Quantum parameters are $4nL$; the classical share includes the n learnable output scales.

Config	Qubits n	Layers L	Quantum	Classical	Total	Ratio to classical
classical	—	—	0	4353	4353	1.00
narrow	2	2	16	2405	2421	0.56
baseline	4	2	32	2537	2569	0.59
wide	6	2	48	2669	2717	0.62
deep	4	4	64	2537	2601	0.60

VQC returns n Pauli- Z expectation values, a learnable per-qubit scale is applied, and a classical post-processor $n \rightarrow 32 \rightarrow 32 \rightarrow 1$ produces $\hat{T}_\theta$.

Circuit. The features are encoded by angle embedding (one R_X per qubit). Each of the L layers applies R_Z, R_X on every qubit, a ring of CNOT gates $i \rightarrow (i + 1) \bmod n$, then R_X, R_Z on every qubit, giving $4nL$ trainable quantum parameters. The output is $\langle Z_i \rangle$ for each qubit. The circuit follows the design of Farea et al. [7]. Gradients are obtained by backpropagation on the `default.qubit` simulator (PennyLane [3]).

3.3 Configurations

Table 3 lists the four hybrid configurations. *Wide* increases the qubit count of the baseline from 4 to 6 at fixed depth; *narrow* reduces it to 2; *deep* keeps 4 qubits and doubles the depth to 4 layers. Figure 2 shows an exemplar circuit for each.

4 Experimental Setup

4.1 Timed stages

We time four cumulative stages (defined and motivated in Section 2.2) on the same batch of collocation points:

1. **forward**: evaluate $\hat{T}_\theta(x)$, no derivatives – a plain prediction;
2. **first_grad**: the forward pass plus $\nabla_x \hat{T}_\theta$;
3. **second_grad**: the forward pass, $\nabla_x \hat{T}_\theta$ and $\partial^2 \hat{T}_\theta / \partial z^2$ – everything the PDE residual r_θ needs;

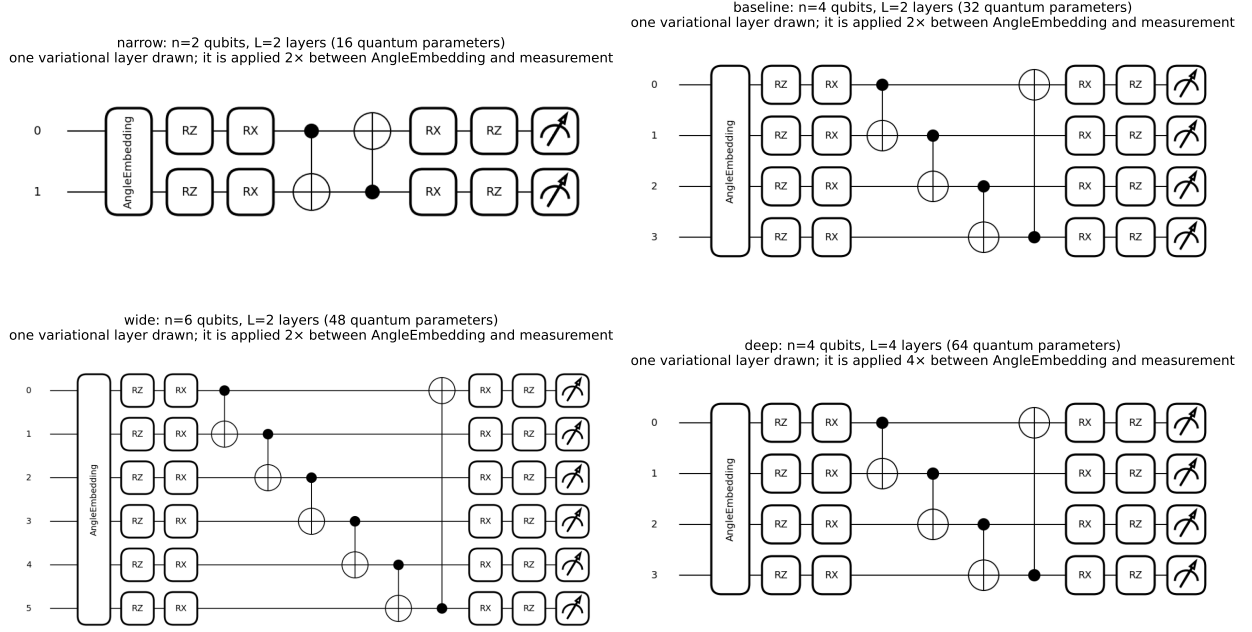


Figure 2: Exemplar circuits of the four hybrid configurations (narrow, baseline, wide, deep).

4. **param_grad:** the `second_grad` graph plus one further backward pass of the PDE-residual loss to the parameters, $\partial \mathcal{L}_{\text{PDE}} / \partial \theta$ (Section 2.5) – approximately the PDE term’s share of one optimiser step.

The stages are cumulative, so a ratio such as the *second-over-forward ratio* is the cost of everything needed for the PDE residual relative to a plain evaluation, not the cost of the second derivative alone. `param_grad` covers only the PDE term of $\mathcal{L}$; the boundary, initial-condition and data terms add further backward passes that we do not separately time, so a full training step costs somewhat more than `param_grad`.

4.2 Size-matched classical controls

For each hybrid configuration we also time a purely classical network in which the VQC is replaced by a $\tanh$ layer of the same width n , giving a network with exactly the hybrid’s classical parameter count (`ctrl_n2`, `ctrl_n4`, `ctrl_n6`; *deep* shares its control with *baseline*, since both have $n = 4$). Comparing a hybrid to its matched control, rather than only to the full classical baseline, separates the cost of *having fewer parameters* from the cost of *simulating the quantum circuit*: the matched control has the hybrid’s parameter count but no circuit to simulate.

4.3 Protocol

Each stage is timed over 5–20 repetitions per round (adaptively chosen so a round takes about a fixed time budget) after 3 warm-up calls, on a batch of 2540 collocation points sampled uniformly in $[0, 2] \times [0, 5]$, matching the training batch size; batch size is itself swept from 100 to 5000 in a separate experiment (Section 6.5). Before the first timed section, a 120s settle phase of untimed work on the drift-check cases brings CPU clocks and thread scheduling to a steady state; this was added after an earlier run showed the hybrid configurations running up to 9% faster at the end than

at the start, consistent with CPU warm-up rather than background load. Cases (configuration $\times$ seed) are interleaved in a shuffled order within each round, so that slow drift of the machine affects all cases alike rather than biasing whichever ran last. Network weights, batch data and circuit parameters are redrawn for each of 3 independent seeds (0, 1, 2); we report the mean over seeds of the per-seed mean, and the sample standard deviation across seeds. Backends are `default.qubit` (vectorised over the batch) and `lightning.qubit`, compared on the circuit alone (Section 6.6).

A drift check re-times a subset of the *main* cases (*classical*, *baseline*, *wide*) at the end of the run and compares them against the same cases at the start. Judged against all cells, this flagged deviations above 10% on every attempt, but the failures were driven almost entirely by the classical model’s sub-millisecond stages, where a handful of scheduling-jitter outliers move the mean by 10–30% while the minimum barely changes. The check now uses the median ratio of only the cells that take at least 5 ms in *main*; the faster, jitter-dominated cells are still reported but excluded from the verdict. Under this rule, the sixth independent run (`win_py314_run6`) – taken with the machine on a high-performance power plan, on AC power, with background applications closed, and after the settle phase above – passes with a largest deviation of 4.2% (*wide second_grad*), the smallest of six independent runs, and its numbers are the ones reported in this paper.

5 Results I: Parameter Count

Table 3 (Section 3.3) already lists each configuration’s classical and quantum parameter count; Table 4 restates the same counts as the two quantities that matter for the parameter-count claim – the quantum branch’s share of the total, and the reduction relative to the classical baseline.

Table 4: Quantum share of the total parameter count, and the reduction relative to the classical baseline (4353 parameters). Quantum share is $4nL$ divided by total parameters.

Config	Quantum share	Reduction vs. classical
narrow	0.66%	44.4%
baseline	1.25%	41.0%
wide	1.77%	37.6%
deep	2.46%	40.2%

Across the four hybrid configurations, the VQC itself contributes only 16–64 trainable parameters – at most 2.46% of the network’s total. Total parameter count nonetheless falls by 37.6–44.4% relative to the classical baseline, because the classical pre- and post-processor around the VQC is already narrower than the classical baseline’s five hidden layers of width 32 (Section 3). The headline parameter-count advantage of the hybrid models is therefore almost entirely a property of the classical architecture chosen around the quantum layer, not of the quantum layer itself. This is what motivates the size-matched controls of Section 4.2: comparing a hybrid only to the classical baseline would credit the VQC with parameter savings it did not produce.

6 Results II: Hardware Timing

Numbers in this section are from `win_py314_run6`, the run whose drift check passed with the smallest deviation of the six independent runs (Section 4); all times are milliseconds, mean over 3 seeds $\pm$ sample standard deviation across seeds, on the default batch of 2540 collocation points unless stated otherwise.

6.1 Per-configuration timing

Table 5 gives the four timed stages for the classical baseline, the four hybrid configurations and their size-matched controls (Section 4.2).

Table 5: Stage timing at the default batch size (ms, mean $\pm$ std across 3 seeds).

Config	forward	first_grad	second_grad	param_grad
classical	0.53 ± 0.01	0.95 ± 0.09	1.61 ± 0.01	4.06 ± 0.07
narrow	7.60 ± 0.13	11.25 ± 0.16	21.67 ± 0.45	42.15 ± 1.16
baseline	16.08 ± 0.60	30.44 ± 0.20	62.65 ± 0.84	126.4 ± 0.2
wide	29.88 ± 0.13	80.93 ± 1.62	153.7 ± 0.7	394.3 ± 6.2
deep	26.34 ± 0.47	52.54 ± 1.09	97.30 ± 1.06	203.7 ± 1.3
ctrl_n2	0.70 ± 0.00	1.09 ± 0.03	1.90 ± 0.01	4.54 ± 0.04
ctrl_n4	0.70 ± 0.01	1.11 ± 0.04	1.95 ± 0.02	4.58 ± 0.04
ctrl_n6	0.71 ± 0.01	1.14 ± 0.01	2.01 ± 0.13	4.65 ± 0.07

Table 6 converts these into slowdowns relative to the classical baseline and relative to each hybrid’s size-matched control. The comparison to the matched control is consistently smaller: it isolates the cost of *simulating the circuit* from the cost of *having fewer classical parameters* that Section 5 already accounts for – e.g. wide’s param_grad is $97.1\times$ the classical baseline but $84.9\times$ its own matched control, so about 13 percentage points of the naive ratio comes from the parameter-count difference rather than the circuit.

Table 6: Slowdown relative to the classical baseline and to each hybrid’s size-matched control (narrow $\rightarrow$ ctrl_n2, baseline and deep $\rightarrow$ ctrl_n4, wide $\rightarrow$ ctrl_n6).

Config	vs. classical				vs. matched control			
	fwd	1st	2nd	param	fwd	1st	2nd	param
narrow	$14.3\times$	$11.9\times$	$13.5\times$	$10.4\times$	$10.9\times$	$10.3\times$	$11.4\times$	$9.3\times$
baseline	$30.3\times$	$32.1\times$	$38.9\times$	$31.1\times$	$23.0\times$	$27.4\times$	$32.1\times$	$27.6\times$
wide	$56.3\times$	$85.4\times$	$95.5\times$	$97.1\times$	$42.2\times$	$70.7\times$	$76.4\times$	$84.9\times$
deep	$49.7\times$	$55.5\times$	$60.5\times$	$50.2\times$	$37.7\times$	$47.3\times$	$49.8\times$	$44.4\times$

Second-over-forward and param-over-forward ratios. Because the four stages are cumulative (Section 2.2), the ratio of a later stage to **forward** measures the extra cost of checking or updating the network once a prediction already exists. This ratio grows with model size: for **second_grad** it is 3.03 for the classical baseline, 2.85–3.90 for narrow/baseline, and 5.14 for wide; for **param_grad** it is 7.65 for classical, rising to 13.20 for wide. The extra backward passes needed for the PDE residual and its parameter gradient are therefore not a fixed overhead – they compound with the size of the network they run through.

6.2 Quantum share of hybrid time

Module hooks on `HybridQNN.quantum` (Section 4) split each hybrid stage’s time into the time spent inside the VQC and the time spent in the rest of the network – the classical pre/post-processor and PyTorch’s own autograd bookkeeping. Table 7 reports this split for the four hybrid configurations at the default batch size.

Table 7: Quantum-circuit share of each hybrid stage’s total time (%), at the default batch size.

Config	forward	first_grad	second_grad	param_grad
narrow	90%	86%	58%	44%
baseline	95%	91%	65%	44%
wide	97%	95%	66%	42%
deep	97%	94%	69%	46%

The quantum circuit accounts for 90–97% of **forward** time, but only 42–46% of **param_grad** time: each further backward pass adds classical autograd bookkeeping on top of the (comparatively fixed) circuit-simulation cost, so the classical side’s *share* grows even though the classical network itself did not change. This split is stable across the qubit, depth and batch sweeps below (88–100% at **forward**, 38–51% at **param_grad**, in the corresponding per-sweep breakdowns), so it is a property of the stage, not of any one configuration.

6.3 Qubit sweep

Table 8 times a 2-layer circuit as qubit count n increases from 2 to 12, holding everything else fixed. Second- and parameter-gradient timing is omitted above 8 qubits, where a single forward pass already exceeds half a second and repeated backward passes become impractical to collect within the benchmark’s time budget.

Table 8: Qubit sweep, 2 layers (ms).

n qubits	forward	first_grad	second_grad	param_grad
2	7.44 ± 0.09	12.89 ± 0.50	21.05 ± 0.49	41.09 ± 0.24
4	16.20 ± 0.26	32.35 ± 0.34	62.77 ± 1.20	127.0 ± 0.6
6	31.58 ± 0.28	81.43 ± 1.49	147.6 ± 0.5	386.1 ± 2.3
8	121.5 ± 0.3	332.3 ± 3.0	778.8 ± 0.5	2229.5 ± 3.1
10	681.6 ± 6.0	—	—	—
12	3176.4 ± 5.2	—	—	—

A log2-linear fit gives a per-qubit growth factor of $1.84\times$ (forward), $1.71\times$ (first_grad), $1.79\times$ (second_grad) and $1.92\times$ (param_grad) – consistent with the exponential cost of statevector simulation (Section 2.6), and slightly steeper for the later, more heavily backpropagated stages.

6.4 Circuit depth sweep

Table 9 times a 4-qubit circuit as depth L increases from 1 to 8 layers.

Table 9: Depth sweep, 4 qubits (ms).

L layers	forward	first_grad	second_grad	param_grad
1	10.83 ± 0.11	19.27 ± 0.13	40.18 ± 0.30	86.73 ± 0.15
2	15.91 ± 0.27	30.16 ± 0.88	61.32 ± 0.08	125.4 ± 1.2
4	25.86 ± 0.52	49.04 ± 0.79	94.75 ± 0.55	201.8 ± 3.3
8	45.42 ± 0.48	91.28 ± 1.05	161.7 ± 0.8	354.9 ± 2.1

Unlike the qubit sweep, depth adds cost roughly linearly: a linear fit gives 4.94 ms per added layer for `forward`, 10.24 ms for `first_grad`, 17.15 ms for `second_grad` and 38.30 ms for `param_grad`, matching each layer contributing an equal-sized block of gates to simulate and differentiate through, rather than doubling the state space the way an added qubit does.

6.5 Batch-size sweep

Table 10 times the baseline configuration (4 qubits, 2 layers) as batch size increases from 100 to 5000, holding the network fixed; 2540 is the training batch size used elsewhere in this paper.

Table 10: Batch-size sweep, baseline configuration (ms).

Batch	forward	first_grad	second_grad	param_grad
100	11.63 ± 0.25	16.40 ± 0.64	25.04 ± 0.47	49.74 ± 0.85
500	13.56 ± 0.18	20.52 ± 0.77	34.89 ± 0.41	67.89 ± 0.43
1000	14.22 ± 0.03	24.47 ± 0.50	43.38 ± 0.44	89.93 ± 1.18
2540	16.03 ± 0.08	31.41 ± 0.13	63.14 ± 1.13	125.7 ± 0.6
5000	18.91 ± 0.10	42.01 ± 0.78	81.60 ± 0.51	183.8 ± 0.6

A log-log fit of time against batch size gives an exponent of 0.12 (forward) rising to 0.33 (param_grad): time grows sub-linearly with batch size at every stage, since `default.qubit` vectorises the circuit over the batch dimension rather than looping over it (Section 6.6), but the growth exponent is larger for the later stages because their added backward passes include classical operations that scale more directly with batch size.

6.6 Backend comparison

Table 11 compares the circuit-only forward pass on `default.qubit` against `lightning.qubit`, PennyLane’s C++ statevector backend, across the four configurations and the batch sweep.

Table 11: Circuit-only forward pass: `default.qubit` vs. `lightning.qubit` (ms).

Case	default.qubit	lightning.qubit	lightning / default
narrow	6.41 ± 0.25	719.7 ± 1.7	$112.3\times$
baseline	14.16 ± 0.27	1417.5 ± 6.1	$100.1\times$
wide	27.46 ± 0.53	2073.3 ± 0.6	$75.5\times$
deep	22.73 ± 0.38	2502.7 ± 6.3	$110.1\times$
batch100	8.14 ± 0.21	84.23 ± 1.66	$10.3\times$
batch500	9.97 ± 0.36	293.1 ± 1.8	$29.4\times$
batch1000	11.80 ± 0.18	551.3 ± 2.7	$46.7\times$
batch2540	14.37 ± 0.16	1416.5 ± 2.7	$98.6\times$
batch5000	16.53 ± 0.29	2733.3 ± 8.2	$165.3\times$

`lightning.qubit` is 75–166 $\times$ *slower* than `default.qubit` here, the opposite of the usual expectation for a dedicated simulator backend, and the gap widens with batch size: a log-log fit gives a batch exponent of 0.18 on `default.qubit` but 0.89 – essentially linear – on `lightning.qubit`. Both backends are called the same way in this benchmark, with the batch passed as one array to a single `QNode` evaluation (Section 4). `default.qubit` natively vectorises this call across the batch dimension, whereas `lightning.qubit` does not support batched (broadcasted) execution

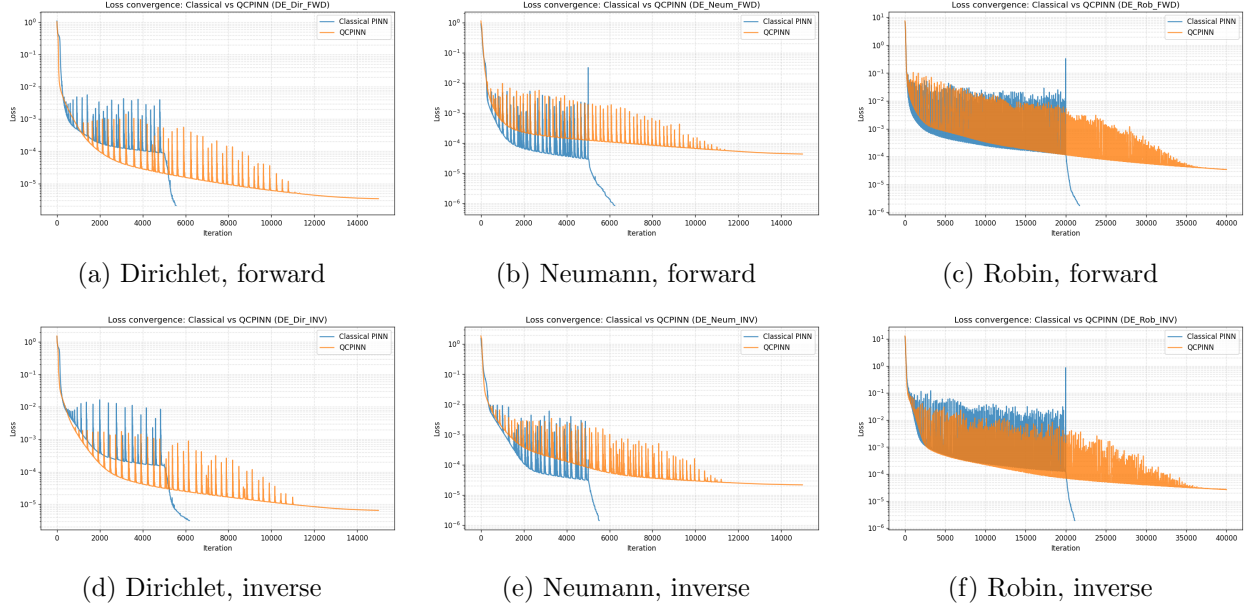


Figure 3: Training loss vs. iteration, classical PINN vs. QCPINN (baseline configuration), for all three boundary conditions and both problem types.

and falls back to one circuit evaluation per sample, so its cost scales linearly with batch size while `default.qubit`'s stays sub-linear (Section 6.5). At the batch sizes used to train this PINN (thousands of collocation points), that per-sample looping – not the circuit simulator's per-gate cost – is what makes `lightning.qubit` the slower choice.

7 Results III: Training Convergence

Figure 3 plots training loss (log scale) against Adam/L-BFGS iteration for the classical baseline and the baseline hybrid configuration ($n_{\text{qubits}} = 4$, $n_{\text{qlayers}} = 2$, Section 4), for all three boundary conditions and both the forward and inverse problem (Section 2). This is a sanity check that the hybrid model reaches a comparable loss at all, independent of the per-step cost differences reported in Section 6.

These curves cover only the baseline hybrid configuration; narrow/wide/deep (Section 4.2) are not yet included. [TODO: re-run with narrow/wide/deep included.]

References

- [1] Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2017.
- [2] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4:043001, 2019.
- [3] Ville Bergholm et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. arXiv:1811.04968, 2018.

- [4] M. Cerezo et al. Variational quantum algorithms. *Nature Reviews Physics*, 3:625–644, 2021.
- [5] Ziv Chen, Gal G. Shaviner, Hemanth Chandravamsi, Shimon Pisnoy, Steven H. Frankel, and Uzi Pereg. Quantum physics-informed neural networks for Maxwell’s equations: Circuit design, “black hole” barren plateaus mitigation, and GPU acceleration. arXiv:2506.23246, 2025.
- [6] Harris Drucker and Yann Le Cun. Improving generalization performance using double back-propagation. *IEEE Transactions on Neural Networks*, 3(6):991–997, 1992.
- [7] Afrah Farea, Saiful Khan, and Mustafa Serdar Celebi. QCPINN: Quantum-classical physics-informed neural networks for solving PDEs. *Machine Learning: Science and Technology*, 6: 045053, 2025. doi: 10.1088/2632-2153/aclc91.
- [8] Lei Han, Changming Dong, Yuli Liu, Huarong Xie, Hongchun Zhang, and Weijun Zhu. Application of physics-informed neural networks in solving temperature diffusion equation of seawater. *Journal of Oceanology and Limnology*, 44(1):1–18, 2026. doi: 10.1007/s00343-025-4348-1.
- [9] Prasad Nimantha Madusanka Ukwatta Hewage, Midhun Chakkravarthy, and Ruwan Kumara Abeysekara. Variational quantum physics-informed neural networks for hydrological PDE-constrained learning with inherent uncertainty quantification. arXiv:2604.09374, 2026.
- [10] Oleksandr Kyriienko, Annie E. Paine, and Vincent E. Elfving. Solving nonlinear differential equations with differentiable quantum circuits. *Physical Review A*, 103:052416, 2021.
- [11] Said Lantigua, Gilson Giraldi, and Renato Portugal. A classical-quantum hybrid architecture for physics-informed neural networks. arXiv:2511.07216, 2025.
- [12] Fong Yew Leong, Wei-Bin Ewe, Si Bui Quang Tran, Zhongyuan Zhang, and Jun Yong Khoo. Hybrid quantum physics-informed neural network: Towards efficient learning of high-speed flows. arXiv:2503.02202, 2025.
- [13] Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.
- [14] Andrea Mari, Thomas R. Bromley, and Nathan Killoran. Estimating the gradient and higher-order derivatives on quantum hardware. *Physical Review A*, 103:012405, 2021.
- [15] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii. Quantum circuit learning. *Physical Review A*, 98:032309, 2018.
- [16] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in PyTorch. In *NIPS 2017 Autodiff Workshop: The Future of Gradient-based Machine Learning Software and Techniques*, 2017.
- [17] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- [18] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [19] Xiang Rao, Yina Liu, and Yuxuan Shen. Quantum-classical physics-informed neural networks for solving reservoir seepage equations. arXiv:2512.03923, 2025.

- [20] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99:032331, 2019.
- [21] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103:032430, 2021.
- [22] Corey Trahan, Mark Loveland, and Samuel Dent. Quantum physics-informed neural networks. *Entropy*, 26(8):649, 2024. doi: 10.3390/e26080649.
- [23] Xiaosi Xu, Simon Benjamin, Jinzhao Sun, Xiao Yuan, and Pan Zhang. A herculean task: Classical simulation of quantum computers. arXiv:2302.08880, 2023.